

Technical Design Document: High-Fidelity Quranic Verification Platform (Architecture Version 2.0: Serverless & Edge-Native)

1. Executive Summary

1.1 The Imperative for Evolution

The digital preservation of Quranic recitation, or *Tajweed*, represents one of the most sophisticated challenges in the intersection of computational linguistics, acoustic signal processing, and real-time cloud architecture. The Quran is not merely a text; it is an oral tradition where the precise duration of vowels (*Madd*), the nasalization of consonants (*Ghunnah*), and the reverberation of stops (*Qalqalah*) are governed by strict, immutable rules established over 1,400 years of scholarship. The preservation of this tradition in a digital verification platform requires a system that functions with the fidelity of a human teacher and the scalability of a global software service.

Version 1.0 of the Verification Platform successfully established a baseline for automated feedback, utilizing a standard RESTful architecture and monolithic containerization. However, as user adoption scaled and the pedagogical requirements deepened from simple error detection to millisecond-precise corrective feedback, the limitations of the legacy architecture became insurmountable. Network latency averaged 800ms—too slow for the cognitive loop of a learner correcting their recitation in real-time. The monolithic Automatic Speech Recognition (ASR) models, deployed on always-on GPU clusters, incurred prohibitive costs during idle periods while struggling to handle the burst traffic characteristic of prayer times and the holy month of Ramadan. Furthermore, the reliance on the React Native "Bridge" for audio visualization resulted in dropped frames and visual jitter, severing the visceral connection between the user's voice and the visual feedback on their screen.

This document serves as the definitive technical blueprint for **Architecture Version 2.0**, a paradigm shift toward a **Serverless & Edge-Native** ecosystem. This re-architecture is not an incremental update but a fundamental rewriting of the platform's DNA. It proposes moving the "fast loop" of audio acquisition, buffering, and high-fidelity visualization entirely to the mobile edge using C++ JSI (JavaScript Interface) bindings, thereby bypassing the JavaScript bridge bottleneck. Simultaneously, the "slow loop" of deep phonetic inference is offloaded to ephemeral, high-performance GPU environments managed by Modal. By transitioning the transport layer from standard HTTP to persistent WebSockets—with a forward-looking roadmap to HTTP/3 WebTransport—and adopting a specialized columnar data strategy for

telemetry, V2.0 targets an end-to-end verification latency of sub-200ms. This capability is the threshold required to transform the platform from a "post-recitation reviewer" into a "real-time digital tutor."

1.2 Core Architectural Objectives

The redesign is driven by four non-negotiable technical objectives:

1. **Latency Reduction to Sub-Perceptual Levels:** The system must mitigate the "cold start" penalties inherent in serverless AI inference. Through advanced container image baking, aggressive model weight caching, and strategic keep-warm pools, the platform targets a P99 inference start time of under 500ms, with steady-state processing latencies well below 200ms.
2. **Edge Fidelity and Zero-Copy Visualization:** The mobile client must deliver 60fps (frames per second) audio visualization on standard hardware and up to 120fps on ProMotion displays. This requires the elimination of JSON serialization for audio data. Architecture V2.0 mandates the use of JSI to enable zero-copy memory sharing between the native C++ audio engine (Oboe) and the graphics engine (Skia), ensuring that visual feedback is mathematically synchronized with auditory input.
3. **Elastic Scalability and Cost Efficiency:** The signaling layer must be decoupled from the inference layer. This separation allows the platform to absorb massive spikes in concurrent connections without the need to over-provision expensive GPU resources. The serverless model transforms the cost structure from a fixed capital expenditure (CapEx) model of renting idle hardware to a pure operational expenditure (OpEx) utility model, paying only for the exact milliseconds of GPU compute utilized.
4. **Acoustic Precision via Tajweed-Aware AI:** The generic ASR approach must be abandoned in favor of a specialized phoneme alignment system. This system will utilize a custom Quranic Phonetic Script (QPS) to distinguish subtle articulatory features that standard IPA (International Phonetic Alphabet) ignores, providing learners with nuanced feedback on specific Tajweed rules rather than generic pronunciation scores.

1.3 Strategic Business and Pedagogical Value

The transition to V2.0 aligns technical capability with strategic business goals. By reducing the marginal cost of verification through serverless efficiencies, the platform can sustain a "freemium" model that is financially viable at scale. Pedagogically, the enhanced fidelity empowers the system to act as a *Muallim* (teacher), diagnosing not just *that* a mistake occurred, but *why*—whether a vowel was held for 300ms instead of the required 600ms, or whether a nasal sound lacked the requisite reverberation. This depth of feedback establishes a competitive moat, differentiating the platform from generic Arabic speech-to-text tools and positioning it as the premier solution for Quranic education.

2. System Architecture

2.1 The Paradigm Shift: From Monolith to Edge-Native Serverless

The traditional three-tier web architecture (Client, Web Server, Database) is ill-suited for real-time, high-bandwidth audio processing. In a typical V1.0 setup, audio is recorded, buffered into a large file (e.g., 5-10 seconds), and uploaded via HTTP POST. The server receives the file, queues it for processing, runs a heavy inference job, and returns a JSON response. This "store-and-forward" model introduces inevitable latency: the user must finish speaking before processing begins.

Architecture V2.0 adopts a **Streaming-First** philosophy. The mobile client acts as an intelligent edge node, performing continuous digital signal processing (DSP) for visualization and voice activity detection (VAD). It streams audio chunks in near real-time (every 100-200ms) to a stateless serverless backend. This effectively pipelines the user's recitation with the AI's verification; the system verifies the beginning of the sentence while the user is still reciting the end.

2.1.1 High-Level Topology Description

The topology of V2.0 is composed of four distinct layers, each optimized for its specific role in the data lifecycle.

1. The Mobile Edge (Client Layer)

This is the user's entry point. It is no longer a "dumb terminal" displaying data from a server. It is a sophisticated signal processing unit.

- **Audio Engine:** We utilize **Google Oboe**, a C++ library that wraps AAudio and OpenSL ES on Android, providing the lowest possible input latency. On iOS, we interface directly with CoreAudio.
- **Bridge Layer:** The standard React Native Bridge is bypassed. A **JSI (JavaScript Interface)** layer exposes C++ host objects to the JavaScript runtime, allowing shared memory access to raw audio buffers.
- **Visualization Engine:** **React Native Skia** is driven by **Reanimated Worklets** on the UI thread, reading directly from the JSI audio buffer to render waveforms and pitch contours at 60fps.
- **Network Layer:** A custom WebSocket client manages the binary stream, implementing application-level ping/pong for connection health and dynamic buffering to handle network jitter.

2. The Gateway Layer (Signaling & Orchestration)

This layer acts as the traffic controller, maintaining persistent connections with mobile clients and dispatching work to the compute layer.

- **Load Balancer:** Google Cloud Load Balancing (GCLB) provides global anycast IP termination, SSL offloading, and DDoS protection.

- **Signaling Server:** A lightweight, highly scalable Node.js or Go fleet running on Cloud Run. It handles authentication, upgrades HTTP requests to WebSockets, manages session state (e.g., "User X is reciting Surah Y"), and routes audio streams to the backend inference queues.

3. The Compute Layer (The Semantic Brain)

This is where the heavy lifting occurs. It is stateless, ephemeral, and elastic.

- **Inference Platform: Modal.com** serves as the execution environment. It provides rapid container provisioning specifically optimized for GPU workloads.
- **GPU Workers:** Auto-scaling containers hosting the **Hafs2Vec** (acoustic model) and **Wav2Vec2** (alignment model). These workers spin up on demand, process a batch of audio, and spin down, ensuring zero cost for idle time.
- **Lifecycle Manager:** A custom control plane within Modal that manages "baking" of container images (pre-loading model weights into RAM) and implementing "keep-warm" strategies to mitigate cold start latency during predictable traffic spikes.

4. The Data Plane (Storage & Analytics)

Data is treated as a first-class citizen, partitioned by "temperature" (access frequency and speed).

- **Hot Storage (State): Redis** clusters manage ephemeral session data, such as the current position in the verse and the immediate feedback queue.
- **Warm Storage (Telemetry): ClickHouse** ingests high-velocity logs, including phoneme-level timing data, confidence scores, and latency metrics. This enables real-time observability and analytics.
- **Cold Storage (Archival):** S3 or Google Cloud Storage buckets store the raw audio files and verified JSON outputs for long-term compliance, legal auditing, and future model retraining.

2.2 Design Principles

The architecture is governed by four foundational design principles that guide every technical decision:

1. Zero-Copy Data Flow

In the mobile client, the cost of copying memory is the enemy of latency. Architecture V2.0 mandates that audio data captured by the microphone must never be serialized (e.g., converted to JSON strings) for internal processing. It must reside in a `SharedArrayBuffer` or a C++ memory block that is accessible by both the audio recording thread (writer) and the visualization thread (reader). This "zero-copy" approach ensures that the visualization is a direct mathematical representation of the audio buffer, free from the garbage collection pauses and serialization overheads of the JavaScript bridge.

2. Stateless Compute

To achieve infinite horizontal scalability, the GPU inference workers must be stateless. They should not retain knowledge of the user's session history beyond the audio context required

for the current inference step. All necessary context (e.g., the target text for alignment) must be passed along with the audio chunk or retrieved from a low-latency external cache like Redis. This allows any available GPU worker to process any chunk of audio, decoupling the scaling of compute resources from the number of active user sessions.

3. Asynchronous Verification with Eventual Consistency

Real-time systems distinguish between "hard real-time" (flight control) and "soft real-time" (media streaming). Our system operates in a soft real-time domain. The visualization of the user's voice must be synchronous and immediate (hard real-time requirement for UX). However, the verification—the red or green highlight indicating correctness—is asynchronous. The architecture accepts an "eventual consistency" model where the correction signal arrives 200-500ms after the audio is spoken. The client UI is designed to handle this delay gracefully, perhaps by showing a "processing" animation or a "trailing" highlight, ensuring the user interface remains fluid even while waiting for the cloud.

4. Observability as Code

In a distributed, serverless system, "it works on my machine" is impossible. Observability must be baked into the code. Every component, from the mobile client's buffer underrun count to the GPU worker's inference time, must emit structured events. We adopt a "high-cardinality" logging strategy, where every event is tagged with user IDs, session IDs, and precise timestamps. This data flows into ClickHouse, allowing us to debug complex distributed failures (e.g., "Why did users in Indonesia experience high latency at 6:00 PM?") that would be invisible in traditional aggregated metrics.

3. Mobile Client Architecture: The Edge-Native Frontier

The mobile client is the primary interface for the reciter and the most critical component for user engagement. In the V1.0 architecture, performance bottlenecks were endemic. The reliance on the standard React Native Bridge to pass audio data from native code to JavaScript for visualization resulted in significant performance penalties. The bridge is asynchronous, serialized, and batched, meaning that high-frequency audio data (e.g., 60 updates per second) would often clog the bridge, delaying other critical messages and causing the UI to "jank" or freeze. V2.0 eliminates this bottleneck by adopting the **New Architecture** (Fabric/TurboModules) patterns and leveraging direct JSI usage.

3.1 Audio Acquisition & Buffer Management: The Physics of Sound

To achieve professional-grade audio latency, we must bypass the standard high-level audio APIs provided by React Native libraries and go straight to the metal.

3.1.1 The Audio Engine: Google Oboe

On Android, audio latency has historically been a fragmented landscape. V2.0 standardizes on

Google Oboe, a C++ library designed to address this fragmentation. Oboe acts as a smart wrapper that automatically selects the best underlying audio API based on the specific device capabilities and Android version.

- **AAudio:** On modern Android devices (Android 8.1+), Oboe utilizes AAudio, a lightweight, high-performance API designed for low-latency audio. AAudio streams communicate directly with the ALSA (Advanced Linux Sound Architecture) driver via memory-mapped buffers, bypassing the Android system mixer and ensuring the shortest possible path from the hardware microphone to the application memory.
- **OpenSL ES:** For older devices, Oboe falls back to OpenSL ES, ensuring backward compatibility while optimizing buffer configurations to minimize latency.
- **CoreAudio (iOS):** On iOS, the system utilizes the RemoteIO Audio Unit, which provides direct access to the audio input and output hardware. The C++ layer abstracts the differences between Oboe and CoreAudio, exposing a unified interface to the upper layers.

3.1.2 The JSI Bridge Implementation

The critical innovation in V2.0 is the **JSI Audio Module**. Instead of passing audio frames through the React Native Bridge, we expose a C++ HostObject directly to the JavaScript runtime. This object holds a reference to the native memory buffer where audio samples are stored.

- **Shared Memory Architecture:** The C++ audio callback function, running on a high-priority real-time thread, writes incoming microphone data into a lock-free ring buffer. This buffer is wrapped in a `jsi::ArrayBuffer` (or a `SharedArrayBuffer` where supported by the JS engine).
- **Direct Access:** The JavaScript thread (specifically the UI thread via `Reanimated`) holds a reference to this C++ object. When a frame update is required for the visualizer, the JS code calls a method on the HostObject to obtain a "view" of the memory. This operation is synchronous and involves no data copying, only pointer arithmetic.
- **Thread Safety:** Audio processing happens on a high-priority thread that must never be blocked (or "preempted") by lower-priority tasks like UI rendering. A standard mutex would be dangerous here, as a UI stall could block the audio thread, causing audio glitches (pops/clicks). Instead, we employ a **Lock-Free Ring Buffer** (atomic operations) to manage the producer-consumer relationship between the audio thread and the UI thread. The audio thread writes to the "head," and the visualization thread reads from the "tail," ensuring that the audio stream remains pristine regardless of UI load.¹

3.2 High-Fidelity Visualization: Seeing the Voice

Visualizing the user's voice (e.g., a waveform or a pitch contour) in real-time is not just eye candy; it is a critical feedback mechanism. It confirms to the user that the system is "listening" and provides immediate cues about volume and rhythm. V2.0 utilizes **React Native Skia**, a high-performance 2D graphics library based on the Skia Graphics Engine (the same engine

used by Chrome and Flutter).

- **The UI Thread Loop:** To ensure smooth animation, the visualization logic must run at the native screen refresh rate (60Hz or 120Hz). We utilize react-native-reanimated to drive this loop. A useFrameCallback hook runs on the UI thread, completely independent of the main JavaScript thread (which handles business logic and networking).
- **The Render Pipeline:**
 1. **Read:** The callback reads the latest audio samples from the JSI AudioBufferHostObject.
 2. **Process:** It computes the Root Mean Square (RMS) for volume or performs a Fast Fourier Transform (FFT) for frequency analysis. This math happens in C++ or highly optimized JavaScript on the UI thread.
 3. **Draw:** The calculated values drive the path data of a Skia Canvas or Path object. Skia renders these paths directly to the GPU using OpenGL or Vulkan.
- **Performance Guarantee:** By decoupling this logic from the main React state updates and network requests, we guarantee a "glitch-free" 60fps experience. Even if the application is busy parsing a large JSON response from the server, the visualization remains buttery smooth because it lives on a separate thread.⁴

Table 1: Mobile Client Rendering Performance Comparison

Feature	Legacy V1.0 (Bridge + SVG)	V2.0 (JSI + Skia + Reanimated)	Improvement
Data Transfer	Serialized JSON strings over Bridge	Zero-copy Shared Memory via JSI	Infinite (O(1) vs O(N))
Frame Rate	Variable (15-45 fps), drops under load	Locked 60/120 fps	Smoothness
CPU Usage	High (due to JSON parsing/GC)	Low (GPU accelerated)	Battery Life
Latency	Visuals lag audio by ~100-200ms	Visuals sync <16ms (1 frame)	Perceived Speed
Thread	Main JS Thread (blocked by logic)	UI Thread (independent)	Responsiveness

3.3 Audio Transport Layer: Streaming Strategy

Once the audio is captured and visualized locally, it must be streamed to the cloud for verification. The choice of transport protocol is the single biggest determinant of network latency.

3.3.1 Protocol Selection: WebSockets vs. WebTransport

V1.0 relied on standard HTTP POST requests for sending chunks of audio. This "short-polling" style introduced the overhead of establishing a new TCP connection (and SSL handshake) for every single chunk, leading to massive latency accumulation. V2.0 adopts a persistent connection model.

The Current Standard: WebSockets

The immediate implementation utilizes binary WebSockets (WSS). WebSockets provide a full-duplex communication channel over a single TCP connection.

- **Pros:** Universally supported by all mobile clients and backend infrastructure (Cloud Run, Modal, AWS). It allows stateful communication, meaning the server knows the "context" of the stream (e.g., user ID, surah ID) without needing it re-sent in every packet header.⁷
- **Cons:** The primary drawback is TCP Head-of-Line (HOL) blocking. Since TCP guarantees order and delivery, if packet #50 is dropped, the operating system will not deliver packets #51-100 to the application until #50 is successfully retransmitted. In real-time audio, this manifests as a sudden pause followed by a "fast-forward" effect, which can desynchronize the verification feedback.⁹

The Future Standard: WebTransport (HTTP/3)

The architecture includes an abstraction layer designed to support WebTransport in the near future. WebTransport is a next-generation API that runs over HTTP/3, which in turn uses QUIC (based on UDP).

- **Mechanism:** QUIC streams are independent. If a packet in one stream is lost, it does not block the delivery of packets in other streams. Furthermore, WebTransport supports "datagrams"—unreliable, out-of-order packets. For real-time audio, this is ideal: if an audio packet is lost, it is often better to skip it and play silence than to pause the entire stream to wait for it.¹⁰
- **Implementation:** The client will attempt to establish a WebTransport session. If the network blocks UDP (common in corporate firewalls), it gracefully falls back to WebSockets. This "progressive enhancement" strategy ensures the best possible performance for users on modern networks (5G/Fiber) while maintaining compatibility for all.¹²

Table 2: Transport Protocol Capabilities

Feature	HTTP/1.1 (REST)	WebSockets (TCP)	WebTransport (QUIC/UDP)
Connection	Short-lived (per request)	Persistent (stateful)	Persistent (stateful)
Head-of-Line Blocking	Yes (Severe)	Yes (TCP level)	No (Stream independence)
Handshake Overhead	High (Repeated)	Once (Initial)	Lowest (0-RTT Resume)
Reliability	Guaranteed	Guaranteed	Configurable (Reliable or Datagram)
Ideal Use Case	File Uploads	Chat / Notifications	Real-time Media / Gaming

3.3.2 Backpressure and Flow Control

Network conditions are volatile. A user might move from WiFi to a spotty 4G connection while reciting. The client implements a "backpressure" mechanism. It monitors the buffered amount of data in the WebSocket outgoing queue.

- **Healthy:** Buffer is empty or low. Send high-quality audio (e.g., Opus 32kbps).
- **Congested:** Buffer is growing. The client dynamically switches to a lower bitrate or drops non-essential packets (if using WebTransport datagrams). It may also pause the "Visualizer" to save CPU cycles for the network stack. This adaptive behavior prevents the "spiral of death" where a lagging network causes the app to crash due to memory exhaustion.

4. Cloud Infrastructure: Serverless & GPU-Native

The backend infrastructure acts as the "brain" of the platform. It must ingest the audio streams, decode the phonetics, check the Tajweed rules, and return feedback. V2.0 transitions from a generic container orchestration model (Kubernetes) to a specialized **Serverless GPU** model using **Modal.com**.

4.1 Compute Platform: The Choice of Modal.com

The decision to use Modal over AWS Lambda or Google Cloud Run (standard) is pivotal and driven by the unique requirements of AI inference.

The Cold Start Problem

In serverless computing, a "cold start" occurs when a function is invoked after a period of inactivity. The provider must provision a machine, download the container image, start the container, and load the application. For a Python-based AI app, this also involves loading massive model weights (e.g., PyTorch models can be 2-3GB) into GPU VRAM.

- **AWS Lambda:** Typically optimized for CPU tasks. GPU support is limited, and loading a 3GB model can take 30-60 seconds—completely unacceptable for a real-time app.
- **Modal's Innovation:** Modal utilizes a proprietary container runtime and a distributed file system designed for large weights. It supports "memory snapshotting," where the initialized state of the application (after loading weights) is saved to disk. When a new function is triggered, Modal restores this snapshot into memory in sub-second time. This reduces the cold start penalty from ~45 seconds to <1 second, making serverless viable for real-time interaction.¹³

Developer Experience & Infrastructure-as-Code

Modal allows defining the infrastructure directly in the application code.

Python

```
# Conceptual Modal Infrastructure Definition
import modal

app = modal.App("quran-verification")
image = modal.Image.debian_slim().pip_install("torch",
"torchaudio").run_commands("download_models.sh")

@app.function(
    gpu="A10G",
    timeout=600,
    keep_warm=5 # Maintain 5 warm containers during prayer times
)
async def verify_recitation(audio_stream):
    ...
```

This "Infrastructure-as-Code" approach simplifies the deployment pipeline. There is no separate Terraform state or YAML configuration hell. The infrastructure definition lives

alongside the business logic, ensuring they are always in sync.¹⁵

4.2 The Inference Pipeline: Ephemeral & Elastic

The inference function running on Modal is designed as an asynchronous generator or a WebSocket handler.

1. Ingestion & VAD

The function receives a stream of audio bytes. Before invoking the heavy GPU model, a lightweight, CPU-based Voice Activity Detection (VAD) (e.g., Silero VAD) runs on the input. This acts as a gatekeeper. If the user is silent or there is only background noise, the data is discarded, preventing expensive GPU cycles from being wasted on processing silence.

2. The Forward Pass (Hafs2Vec)

Once speech is detected, the audio is buffered into a suitable window (e.g., 2-3 seconds for context) and fed into the Hafs2Vec model (a fine-tuned Wav2Vec2 XLSR-53).

- **Hardware:** We utilize NVIDIA A10G or T4 GPUs. T4s are cost-effective for inference but have lower memory bandwidth. A10Gs offer higher throughput. We dynamically select the GPU type based on the current queue depth—using cheaper T4s for background tasks and faster A10Gs for real-time user sessions.
- **Quantization:** To further speed up inference and reduce VRAM usage, the model is quantized to **FP16** (half-precision) or **INT8** (8-bit integer) using TensorRT or ONNX Runtime. This can double the inference speed with negligible loss in accuracy for phonetic recognition.¹⁷

3. Alignment & Decoding

The model outputs a probability matrix (logits) representing the likelihood of each phoneme at each time step. This output is then aligned to the expected Quranic text using CTC Segmentation (detailed in Section 5).

4. Response Strategy

The result (e.g., "Verse 3, Word 2: Correct", "Verse 3, Word 4: Elongation Error") is serialized to JSON and sent back via the established WebSocket. The function remains alive for the duration of the user's session (or until a timeout), maintaining the "context" (e.g., the hidden state of the Transformer model) to improve the accuracy of subsequent predictions.

4.3 Scalability & Cost Modeling

Traditional GPU clusters require provisioning for peak load. If you expect 10,000 users during Taraweeh prayers in Ramadan, you must pay for enough GPUs to handle that load, even if the system is empty at 3 AM. This leads to massive waste.

The Serverless Economics

Modal's architecture allows scaling from 0 to N GPUs based on the actual demand.

- **Concurrency:** We configure the Modal function to handle multiple concurrent requests per container (e.g., `allow_concurrent_inputs=10`) if the GPU memory allows batching. If a single A10G can process 10 audio streams in parallel without lagging, the cost per user

drops by 90%.¹⁸

- **Billing:** We pay only for the seconds the GPU is active. If a user pauses to breathe or read the Tafsir, the GPU goes idle (or switches to another user), and the billing stops or reduces.

Timeout Management & Keep-Alives

WebSockets on serverless platforms often have execution time limits. While Modal supports long-running execution (up to 24 hours), practical limitations often exist in the gateway layer (e.g., AWS API Gateway limits connections to 2 hours).

- **Reconnection Logic:** The client is designed to automatically reconnect if the session exceeds 60 minutes. This is transparent to the user; the visualizer might freeze for 100ms, but the session state is restored from the Redis cache.¹⁹
- **Keep-Warm Pools:** During known peak times (e.g., the hour before Maghrib in Ramadan), we programmatically increase the keep_warm count on Modal. This ensures a pool of ready-to-go containers exists to absorb the initial wave of users, preventing them from seeing "Loading..." spinners.¹³

5. AI & Phonetic Verification Logic: The Science of Tajweed

This section details the core intellectual property of the platform: the capability to verify Quranic recitation with high fidelity. Standard Arabic ASR models are trained on Modern Standard Arabic (MSA) or dialects like Egyptian or Levantine. These models fail to capture Tajweed nuances. They might recognize the word "Qala" (He said), but they cannot distinguish between a "Qala" with a 2-count vowel (correct) and a "Qala" with a 4-count vowel (incorrect).

5.1 Model: Hafs2Vec (Wav2Vec2 Adaptation)

We introduce **Hafs2Vec**, a specialized acoustic model.

- **Base Architecture:** We utilize **Wav2Vec 2.0 (XLSR-53)** as the foundation. This model is pre-trained on 53 languages, giving it a rich "universal phonetic" understanding. It learns to map raw audio waveforms to latent speech representations.²¹
- **Fine-Tuning Data:** The model is fine-tuned on the **QuranMB** and proprietary datasets. These datasets consist of thousands of hours of professional recitations (*Murattal*) annotated with precise phoneme boundaries, as well as "student" recitations containing common errors.
- **Objective Function:** The training objective minimizes the **Phoneme Error Rate (PER)** rather than the Word Error Rate (WER). In Tajweed, getting the word right but the phoneme wrong (e.g., pronouncing a thin 't' instead of a thick 'T') is a critical failure.

5.2 Quranic Phonetic Script (QPS) vs. IPA

The International Phonetic Alphabet (IPA) is the standard for linguistics, but it lacks specific symbols for Tajweed phenomena. For example, IPA has a symbol for n (noon), but it does not have separate symbols for the different *levels* of nasalization required in Tajweed (Izhar, Ikhfa, Idgham).

The QPS Solution

We map the model's output layer to a custom dictionary called Quranic Phonetic Script (QPS).

- **Granularity:** QPS expands the vocabulary.
 - Standard Arabic: n
 - QPS: n_izhar (Clear), n_ikhfa (Hidden/Nasal), n_idgham_ghunnah (Merged with nasal), n_idgham_bila_ghunnah (Merged without nasal).
- **Implication:** This allows the verification engine to detect highly specific errors. If the text contains a "Noon Sakinah" followed by "Ba" (Rule of Iqlab), the expected token is m_iqlab. If the model predicts n_clear, the system generates a precise error message: "You pronounced the Noon clearly, but you should have converted it to a Meem sound." This is the difference between a spell-checker and a Tajweed teacher.²¹

5.3 Alignment Algorithms: The Math of Verification

To verify recitation, we must align the *spoken* phonemes (which contain errors) with the *expected* text (the canonical Quran).

5.3.1 Needleman-Wunsch (Global Alignment)

For post-recitation analysis (e.g., "Review your whole session"), we use the **Needleman-Wunsch** algorithm.

- **Logic:** It computes the optimal global alignment between the sequence of predicted phonemes and the sequence of canonical phonemes. It uses a dynamic programming matrix to find the path with the minimum "edit distance."
- **Scoring Matrix:** We utilize a custom scoring matrix that reflects Tajweed pedagogy.
 - Substituting s for z might have a high penalty (distinct letters).
 - Substituting t (thin) for T (emphatic) might have a medium penalty (similar articulation point).
 - Substituting a (short vowel) for aa (long vowel) has a specific penalty related to rhythm.
- **Complexity:** The time complexity is $O(N \times M)$. This is computationally expensive for long chapters (Surahs) but provides the most accurate holistic view of the recitation.²⁴

5.3.2 CTC Segmentation (Real-Time Alignment)

For real-time feedback, Needleman-Wunsch is too slow because it requires the full sequence to be finished before it can start (or complex windowing). We employ **CTC (Connectionist**

Temporal Classification) Segmentation.

- **Logic:** The Wav2Vec2 model outputs a probability distribution for every time step (e.g., every 20ms). It includes a "blank" token between phonemes. CTC segmentation finds the most probable path through this "trellis" that matches the reference text.
- **Robustness:** It is highly robust to "missing words" or "restarts." If a user stutters or repeats a word, CTC segmentation can realign to the current position much faster than global alignment methods, making it ideal for the "karaoke-style" follower in the app.²⁶

5.4 Duration Analysis: The "Madd" Logic

A unique requirement of Tajweed is the strict adherence to vowel duration (*Madd*). In music, rhythm is often relative to tempo (BPM). In Tajweed, it is quantized into "counts" or *Harakat*. One Harakah is roughly the time it takes to open or close a finger.

The Algorithm

1. **Phoneme Timestamping:** The CTC output gives us the start time (t_{start}) and end time (t_{end}) of every phoneme.
 - Example: aa (long vowel) detected from $t=1.2s$ to $t=1.8s$. Duration = 600ms.
2. **Normalization (Tempo Detection):** Since speech rate varies (Hadr vs. Tahqiq), we cannot use fixed millisecond thresholds (e.g., "Long vowel must be > 500ms"). Instead, we calculate the **Average Harakah Duration (AHD)** dynamically based on the user's recent history.
 - $AHD = \text{mean}(\text{duration of surrounding short vowels})$.
 - If the user is reading fast, their short vowels might be 100ms. If slow, 200ms.
3. **Verification Rule:**
 - A "Madd Tabee'i" (Natural Pull) is 2 Harakat. Target Duration $\approx 2 \times AHD$.
 - A "Madd Lazim" (Compulsory Pull) is 6 Harakat. Target Duration $\approx 6 \times AHD$.
4. **Tolerance Windows:**
 - Research suggests that in practice, a "long vowel" is often $\sim 2.4x$ the duration of a short vowel, not exactly $2.0x$. We apply a tolerance window (e.g., $\pm 20\%$) around the target.
 - If the user's duration falls outside this window, the system flags a "Madd Error" (Too Short or Too Long).²⁸

Table 3: Madd Verification Thresholds (Example)

Recitation Style	Short Vowel (AHD)	Madd Tabee'i (Target)	Madd Lazim (Target)	6-Count Acceptance Range

Hadr (Fast)	80 ms	160 ms	480 ms	400 - 560 ms
Tadweer (Medium)	150 ms	300 ms	900 ms	750 - 1050 ms
Tahqiq (Slow)	250 ms	500 ms	1500 ms	1250 - 1750 ms

6. Data Strategy: Telemetry & Observability

In V2.0, "Data is Infrastructure." We move from unstructured text logs to structured, high-cardinality event logging. This data is the lifeblood of the system, used to debug acoustic issues in production and to train future generations of the AI models.

6.1 Telemetry Store: The ClickHouse Advantage

We select **ClickHouse** over traditional row-based databases (Postgres) or search engines (ElasticSearch) for telemetry storage. ClickHouse is a column-oriented OLAP (Online Analytical Processing) database designed for massive write throughput and lightning-fast aggregation queries.

Why Columnar?

- **Compression:** Telemetry data is repetitive. The user_id column might have the same value for 1,000 consecutive rows (events in one session). Columnar storage allows aggressive compression (e.g., Run-Length Encoding or ZSTD), reducing storage costs by 90% compared to JSON blobs.³⁰
- **Aggregation Speed:** To answer "What is the average latency for users in Egypt?", ClickHouse only reads the country and latency columns. A row-based database would have to read the entire row (including the phoneme data, user metadata, etc.) from disk, which is orders of magnitude slower.³¹

6.1.1 Schema Design Best Practices

The schema is optimized for the specific access patterns of our verification platform.

Table: recitation_events

Column Name	Type	Description
event_time	DateTime64(3)	Timestamp with millisecond precision. Critical for

		sequencing.
session_id	UUID	Unique ID for the recitation session.
user_id	UInt64	Anonymized user identifier.
ayah_id	LowCardinality(String)	The specific verse being recited (e.g., "2:255"). Optimized for repeated strings.
phoneme_predicted	LowCardinality(String)	The phoneme detected by the model (QPS token).
phoneme_expected	LowCardinality(String)	The phoneme from the canonical text.
confidence_score	Float32	The model's probability score (0.0 - 1.0).
duration_ms	UInt16	Duration of the phoneme in milliseconds.
latency_e2e	UInt16	Total time from mic capture to feedback receipt.
error_type	Enum8('None'=0, 'Madd'=1, 'Ghunnah'=2...)	Categorical error type for fast filtering.

Optimization Strategy:

- **Ordering Key:** ORDER BY (user_id, session_id, event_time). This ensures that all events for a specific user session are stored physically contiguously on the disk. This makes "session replay" queries (retrieving all events for one session to visualize the timeline) almost instantaneous.³⁰
- **Partitioning:** PARTITION BY toYYYYMM(event_time). This splits data into monthly chunks. Old months can be easily moved to cheaper "Cold Storage" (e.g., S3-backed MergeTree) or deleted efficiently (dropping a partition is O(1) compared to DELETE FROM table which

is expensive).³³

- **Data Skipping Indices:** We add a minmax index on the latency_e2e column. This allows the database engine to completely skip reading blocks of data where the maximum latency is low, significantly speeding up queries like "Find all sessions with latency > 1000ms".³³

6.2 The Active Learning Feedback Loop

The data pipeline is not a dead end; it is a cycle.

1. **Hard Example Mining:** Queries run nightly on ClickHouse to identify Ayahs with the highest error rates across the user base. Are users consistently failing at Surah Al-Fajr? Is there a specific phoneme transition that the model has low confidence in?
2. **Dataset Augmentation:** Audio samples corresponding to these "hard" regions are flagged. Since we store the raw audio (with user consent) in S3, we can retrieve these specific clips.
3. **Human-in-the-Loop:** These clips are sent to a labeling tool where expert Qaris verify the label. "Did the user *actually* make a mistake, or did the AI hallucinate?"
4. **Retraining:** These corrected samples ("Hard Negatives") are added to the training set. The Hafs2Vec model is fine-tuned on this new data, improving its robustness against the specific errors users are making. This cycle ensures the AI gets smarter the more it is used.

7. Implementation Roadmap

The transition to V2.0 is complex. We propose a phased approach to mitigate risk.

Phase 1: The Foundation (Months 1-3)

- **Focus:** Cloud Infrastructure & Data.
- **Deliverables:**
 - **Modal.com Setup:** Containerize the Hafs2Vec model. Create "bake" scripts to pre-load weights. Implement the basic WebSocket server in Python (FastAPI/Uvicorn) running on Modal.
 - **Telemetry:** Deploy a ClickHouse cluster (or use ClickHouse Cloud). Define the schemas and set up the ingestion pipeline (Vector or Fluentd) to move logs from Modal to ClickHouse.
 - **Milestone:** A CLI tool that sends a pre-recorded WAV file to the Modal endpoint and receives a JSON stream of phonemes and timing data.

Phase 2: The Edge Client (Months 3-5)

- **Focus:** Mobile Audio & Visualization.
- **Deliverables:**

- **JSI Module:** Write the C++ wrapper for Oboe. Implement the HostObject and shared memory logic. Expose it to the JS runtime.
- **Visualizer:** Build the Skia + Reanimated visualization loop. Test on low-end Android devices to ensure 60fps stability.
- **Networking:** Implement the persistent WebSocket client with buffering and reconnection logic.
- **Milestone:** A "Testflight" app that visualizes microphone input with zero latency and streams it to the Phase 1 backend (even if the backend just echoes it back).

Phase 3: Integration & Intelligence (Months 5-6)

- **Focus:** Connecting the Brain to the Body.
- **Deliverables:**
 - **Full Pipeline:** Connect the Phase 2 client to the Phase 1 backend.
 - **Tajweed Logic:** Implement the CTC Segmentation and "Madd" duration logic in the backend. Calibrate the thresholds based on initial beta user data.
 - **Keep-Warm Optimization:** Analyze usage patterns and configure the Modal keep_warm settings to balance cost vs. latency.
 - **Milestone:** End-to-end verification of Surah Al-Fatiha. User speaks, and within 300ms, the screen highlights the specific word being recited.

Phase 4: Production Readiness (Month 7+)

- **Focus:** Scale, Security, and Future-Proofing.
- **Deliverables:**
 - **Load Testing:** Simulate 10,000 concurrent users using bot scripts to stress-test the Modal scaling triggers and ClickHouse ingestion rates.
 - **WebTransport:** Begin the experimental implementation of HTTP/3 support for compatible clients.
 - **Compliance:** Finalize GDPR audits. Ensure data residency rules are met (e.g., storing EU user audio in EU S3 buckets).
 - **Launch:** Public rollout of V2.0.

8. Conclusion

Architecture Version 2.0 represents the maturation of the Quranic Verification Platform from an experimental prototype to a robust, scalable, and pedagogical instrument. By embracing **Edge-Native** technologies (C++ JSI, Skia) on the client, we solve the visualization latency challenge, reconnecting the user with their own voice. By adopting **Serverless GPU** (Modal) on the backend, we solve the economic challenge of scaling AI inference, moving from a rigid server-based cost structure to a flexible utility model. By grounding the logic in **Tajweed-specific phonetics** (Hafs2Vec, QPS) and **Time-Series Data** (ClickHouse), we ensure the platform is not just a tech demo, but a scientifically rigorous tool for preserving the

oral tradition of the Quran.

The technical risks—primarily the complexity of maintaining C++ code in the mobile client and the reliance on network stability—are significant but manageable through rigorous engineering practices and graceful degradation strategies. The potential reward is transformative: offering every student, regardless of their location or access to a human scholar, a real-time, high-fidelity digital tutor. This architecture leverages the bleeding edge of the 2026 tech stack to solve a 1,400-year-old pedagogical challenge, preserving the past with the tools of the future.

Works cited

1. JSI, C++ & React Native Old Architecture — A Practical Hands-On Guide | by Rohit Mondal, accessed on January 9, 2026, <https://rohitmondal929.medium.com/jsi-c-react-native-old-architecture-a-practical-hands-on-guide-8d8c3617a4bb>
2. How to share an ArrayBuffer with c++ and js using JSI without copying? #1264 - GitHub, accessed on January 9, 2026, <https://github.com/facebook/hermes/discussions/1264>
3. Performance degradation with large objects and arrays · Issue #5781 · software-mansion/react-native-reanimated - GitHub, accessed on January 9, 2026, <https://github.com/software-mansion/react-native-reanimated/issues/5781>
4. @sheunglaili/react-native-skia-video - NPM, accessed on January 9, 2026, <https://www.npmjs.com/package/%40sheunglaili%2Freact-native-skia-video>
5. Shared Values | React Native Reanimated, accessed on January 9, 2026, <https://docs.swmansion.com/react-native-reanimated/docs/2.x/fundamentals/shared-values/>
6. Skia: Game Changer for React Native in 2026 | ExpertAppDevs.Com | Medium, accessed on January 9, 2026, <https://medium.com/@expertappdevs/skia-game-changer-for-react-native-in-2026-f23cb9b85841>
7. WebSocket vs WebRTC for Real-Time Audio Communication | by Inseong Han | Medium, accessed on January 9, 2026, <https://medium.com/@inssa1102/websocket-vs-webrtc-for-real-time-audio-communication-d3b05edb5f41>
8. If WebRTC can do Video, Audio, and Data, why do I need WebSocket? - Stack Overflow, accessed on January 9, 2026, <https://stackoverflow.com/questions/18799364/webrtc-vs-websocket-if-webrtc-can-do-video-audio-and-data-why-do-i-need-webs>
9. Support for WebTransport over HTTP/3 QUIC · OAI OpenAPI-Specification · Discussion #4343 - GitHub, accessed on January 9, 2026, <https://github.com/OAI/OpenAPI-Specification/discussions/4343>
10. What is Replacing WebSockets? Deep Dive Into WebTransport, HTTP/3 & Real-Time Protocols (2025 Guide) - VideoSDK, accessed on January 9, 2026, <https://www.videosdk.live/developer-hub/websocket/what-is-replacing-websock>

[ets](#)

11. MoQ: Refactoring the Internet's real-time media stack - The Cloudflare Blog, accessed on January 9, 2026, <https://blog.cloudflare.com/moq/>
12. Cloud CDN and Load Balancing support HTTP/3 | Google Cloud Blog, accessed on January 9, 2026, <https://cloud.google.com/blog/products/networking/cloud-cdn-and-load-balancing-support-http3>
13. Cold start performance | Modal Docs, accessed on January 9, 2026, <https://modal.com/docs/guide/cold-start>
14. Introduction | Modal Docs, accessed on January 9, 2026, <https://modal.com/docs/guide>
15. Modal: High-performance AI infrastructure, accessed on January 9, 2026, <https://modal.com/>
16. Images | Modal Docs, accessed on January 9, 2026, <https://modal.com/docs/guide/images>
17. How to reduce GPU cold starts : r/modal - Reddit, accessed on January 9, 2026, https://www.reddit.com/r/modal/comments/1o65a48/how_to_reduce_gpu_cold_starts/
18. Best practices for serverless inference - Modal, accessed on January 9, 2026, <https://modal.com/blog/serverless-inference-article>
19. How can I fix the websocket limits for `Connection duration for WebSocket API: 2 hours`, accessed on January 9, 2026, <https://stackoverflow.com/questions/61971915/how-can-i-fix-the-websocket-limits-for-connection-duration-for-websocket-api-2>
20. Timeouts | Modal Docs, accessed on January 9, 2026, <https://modal.com/docs/guide/timeouts>
21. Hafs2Vec: A System for the Iqra'Eval Arabic and Qur'anic Phoneme-level Pronunciation Assessment - ACL Anthology, accessed on January 9, 2026, <https://aclanthology.org/2025.arabichlp-sharedtasks.62.pdf>
22. elgeish/wav2vec2-large-xlsr-53-arabic - Hugging Face, accessed on January 9, 2026, <https://huggingface.co/elgeish/wav2vec2-large-xlsr-53-arabic>
23. Quran Phonetic Script (QPS) Overview - Emergent Mind, accessed on January 9, 2026, <https://www.emergentmind.com/topics/quran-phonetic-script-qps>
24. Parallel Needleman-Wunsch on CUDA to measure word similarity based on phonetic transcriptions - arXiv, accessed on January 9, 2026, <https://arxiv.org/html/2509.01654v1>
25. How does the Needleman Wunsch algorithm compare to brute force? - Stack Overflow, accessed on January 9, 2026, <https://stackoverflow.com/questions/40615299/how-does-the-needleman-wunsch-algorithm-compare-to-brute-force>
26. A Linear Memory CTC-Based Algorithm for Text-to-Voice Alignment of Very Long Audio Recordings - MDPI, accessed on January 9, 2026, <https://www.mdpi.com/2076-3417/13/3/1854>
27. Building a neural speech recognizer for quranic recitations - BCU Open Access Repository - Birmingham City University, accessed on January 9, 2026,

<https://www.open-access.bcu.ac.uk/13502/1/Building%20a%20neural%20speech%20recognizer%20for%20quranic%20recitations.pdf>

28. Acoustic Analysis of Arabic Short vs. Long Vowels Following Emphatic and Non-Emphatic Consonants by Speakers of Najdi Dialect - King Abdulaziz University Journals, accessed on January 9, 2026,
<https://kauj.researchcommons.org/cgi/viewcontent.cgi?article=1053&context=jah>
29. Improving Automatic Forced Alignment for Phoneme Segmentation in Quranic Recitation - IEEE Xplore, accessed on January 9, 2026,
<https://ieeexplore.ieee.org/iel7/6287639/10380310/10371319.pdf>
30. Schema design | ClickHouse Docs, accessed on January 9, 2026,
<https://clickhouse.com/docs/use-cases/observability/schema-design>
31. Top 15 infrastructure monitoring tools in 2026: a performance and cost-based comparison, accessed on January 9, 2026,
<https://clickhouse.com/resources/engineering/top-infrastructure-monitoring-tools-comparison>
32. Schema Design | ClickHouse Docs, accessed on January 9, 2026,
<https://clickhouse.com/docs/data-modeling/schema-design>
33. Best Practices in ClickHouse, accessed on January 9, 2026,
<https://clickhouse.com/docs/best-practices>